Departamento de Computação
unesp

# Java e Programação Orientada a Objetos

Tipos de programas Java e como compilá-los

Programação Orientada a Objetos em Java

# Aplicação Hello World

- Aplicação *standalone* sem janela
  1. Em um arquivo HelloWorldApp.java escreva:

```
class HelloWorld {
    public static void main(String[] args) {
      System.out.println("Hello World!");
    }
 }
```

# Aplicação Hello World

- Aplicação *standalone* sem janela
  1. Em um arquivo HelloWorldApp.java escreva:

```
class HelloWorld {
    public static void main(String[] args) {
      System.out.println("Hello World!");
    }
 }
```

  2. Para compilar o arquivo, na linha de comandos, usa-se
     - **`javac HelloWorldApp.java`**
     - a compilação irá gerar HelloWorld.class

# Aplicação Hello World

- Aplicação *standalone* sem janela
  1. Em um arquivo HelloWorldApp.java escreva:

```
class HelloWorld {
    public static void main(String[] args) {
      System.out.println("Hello World!");
    }
 }
```

  2. Para compilar o arquivo, na linha de comandos, usa-se
     - **`javac HelloWorldApp.java`**
     - a compilação irá gerar HelloWorld.class
  3. Para executar a classe
     - **`java HelloWorld`**

# Aplicação Hello World

- Aplicação *standalone* sem janela
  1. Em um arquivo HelloWorldApp.java escreva:

```
class HelloWorld {
    public static void main(String[] args) {
      System.out.println("Hello World!");
    }
 }
```

Observe que o nome da classe e o nome do arquivo podem ser diferentes

  2. Para compilar o arquivo, na linha de comandos, usa-se
     - **javac** `HelloWorldApp.java`
     - a compilação irá gerar HelloWorld.class
  3. Para executar a classe
     - **java** **HelloWorld**

# Aplicação Hello World

- Aplicação *standalone* com janela

```
import java.awt.*;
import javax.swing.*;

class HelloWorld extends JFrame {
  public void paint(Graphics g) {
    super.paint(g)
    g.drawString("Hello world!", 40, 100);
  }

  public static void main(String args[]) {
    HelloWorld hello=new HelloWorld();
    hello.setSize(155, 150);
    hello.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    hello.setVisible(true);
  }
}
```

# Programação Orientada a Objetos

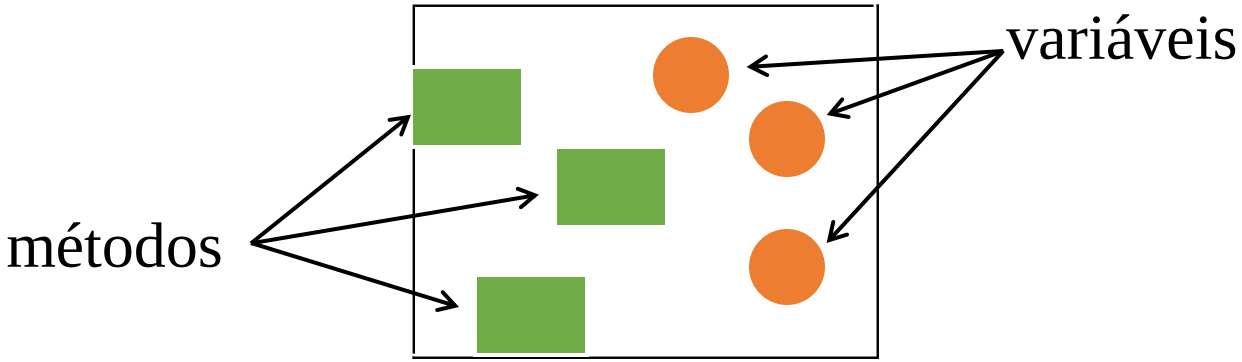
# Programação Orientada a Objetos

- Base para a programação em JAVA

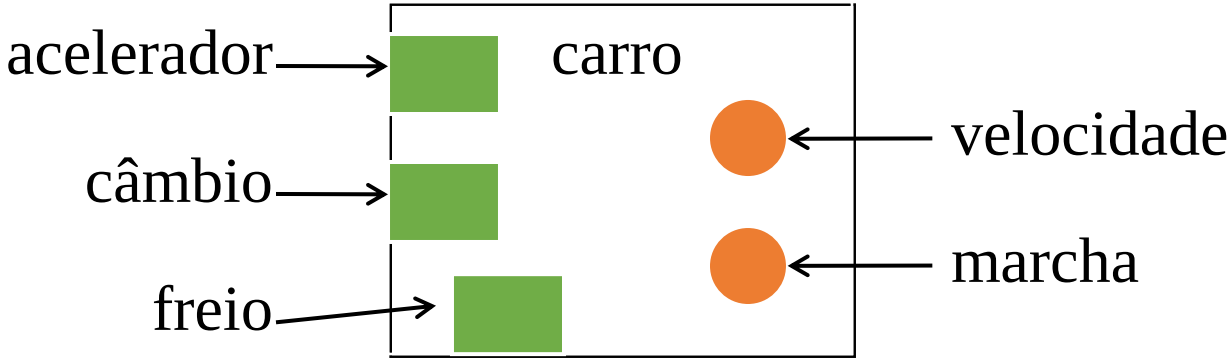
- Principais vantagens da POO:
  - agrupamento de variáveis e rotinas com certo relacionamento;
  - facilita e incentiva a reusabilidade de códigos;
  - melhora a modularidade subdividindo um programa em diversas partes e
  - torna a manutenção mais simples.

# O que é um Objeto?

- Um objeto é um agrupamento de variáveis e rotinas (métodos) relacionados.




exemplo:



# O que são Classes?

- Uma classe é um esquema ou protótipo que define as variáveis e métodos comuns a todos os objetos de um certo tipo
- Classes são “formas” de fazer objetos

- Criação de objetos (instanciação)
  - instâncias de uma classe são os objetos criados a partir desta classe
  - assim, para se criar um objeto (instância) é necessário que a classe desse objeto já esteja definida

# Classes e Instâncias

- Classes tem características similares a *struct* na Linguagem C
  - para usar estas estruturas, em linguagem C, é necessário criar variáveis estáticas ou dinâmicas
  - da mesma forma em POO é necessário alocar memória para acomodar os campos dos objetos (isso significa: criar instância da classe)
  - esses tipos de **campos** são chamados em POO de **variáveis da instância**
  - uma classe pode comportar **rotinas** que são **chamadas de métodos da instância**

# Declarando uma Classe

- Exemplo de uma classe simples

```
class NumeroComplexo {
    float r, i;
}
```

sendo que r e i são variáveis da instância

- Instanciação da classe **`NumeroComplexo`**

```
NumeroComplexo x = new NumeroComplexo();
```

# Declarando uma Classe

- Exemplo de uma classe simples

```
class NumeroComplexo {
    float r, i;
}
```

sendo que r e i são variáveis da instância

- Instanciação da classe **NumeroComplexo**

```
NumeroComplexo x = new NumeroComplexo();
```

- Comparando, em linguagem C:

```
struct NumeroComplexo {
  float r, i;
};
```

- Instanciação

```
struct NumeroComplexo *x = (struct NumeroComplexo*)malloc(
                            sizeof(struct NumeroComplexo));
```

# Declarando uma Classe

- Exemplo de uma classe simples

```
class NumeroComplexo {
    float r, i;
    void atrib(float ar, float ai) {
        r = ar;
        i = ai;
    }
}
```

sendo que atrib é um método da instância.

- Instanciação da classe **`NumeroComplexo`**

```
NumeroComplexo x = new NumeroComplexo();
x.atrib(1.2, 9.8);
```

- A memória será liberada automaticamente

# Instanciação

- A instância da classe **HelloWorld** é referenciada pela variável **hello**

```
import java.awt.*;
import javax.swing.*;

class HelloWorld extends JFrame {
  public void paint(Graphics g) {
    g.drawString("Hello world!", 40, 100);
  }

  public static void main(String args[]) {
    HelloWorld hello=new HelloWorld();
    hello.setSize(155, 150);
    hello.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    hello.setVisible(true);
  }
}
```

Criação da instância

# Exercício

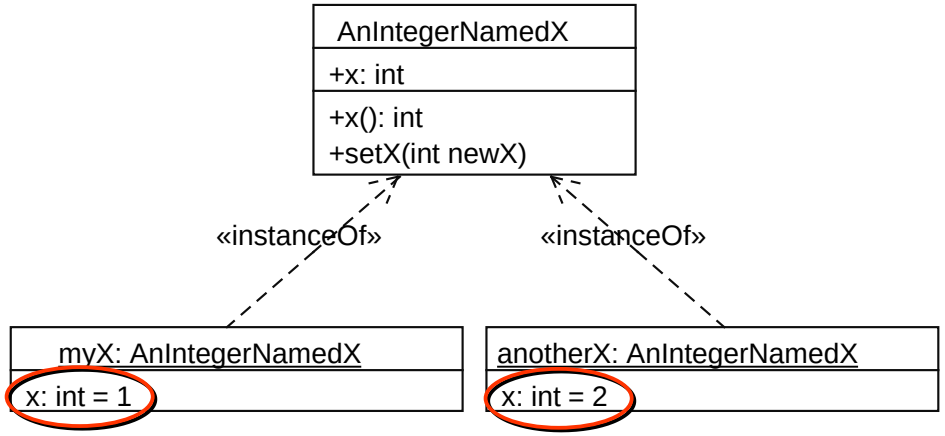
- Compile e execute a classe **`NumeroComplexo`** abaixo e responda:
  - Quais campos e métodos foram definidos?
  - Onde uma instância da classe é criada?
  - O que será apresentado na tela?

```
class NumeroComplexo {
    double r, i;

    void atrib(double ar, double ai) {
        r = ar;
        i = ai;
    }

    public static void main(String s[]) {
        NumeroComplexo x = new NumeroComplexo();
        x.atrib(1.2, 9.8);
        System.out.format("(%.1f + %.1fi)\n", x.r, x.i);
    }
}
```

# Instanciação

```
class AnIntegerNamedX {
   public int x;

   public int x() {
      return x;
   }

   public void setX(int newX) {
     x = newX;
   }
}
  . . .
  AnIntegerNamedX myX = new AnIntegerNamedX();
  AnIntegerNamedX anotherX = new AnIntegerNamedX();
  myX.setX(1);
  anotherX.x = 2;
  System.out.println("myX.x = " + myX.x());
  System.out.println("anotherX.x = " + anotherX.x());

  . . .
```

# Instanciação




```
class AnIntegerNamedX {
   public int x;

   public int x() {
      return x;
   }

   public void setX(int newX) {
     x = newX;
   }
}
   . . .
   AnIntegerNamedX myX = new AnIntegerNamedX();
   AnIntegerNamedX anotherX = new AnIntegerNamedX();
   myX.setX(1);
   anotherX.x = 2;
   System.out.println("myX.x = " + myX.x());
   System.out.println("anotherX.x = " + anotherX.x());

   . . .
```

No vídeo:

```
myX.x = 1
anotherX.x = 2
```

# Exercício usando Classe

- Defina uma classe chamada Pessoa com os atributos (campos): nome, endereco, telefone.
  - use o tipo **String** para definir os campos.

- A classe deve ter métodos (funções) para obter os valores dos campos e para alterá-los.

- Escreva uma função main() e crie uma instância da classe Pessoa.

# Continua ...

# Na aula passada aprendemos

- Declarar uma classe, criar instâncias e usar campos

```
class NumeroComplexo {
    double r, i;

    void atrib(double ar, double ai) {
        r = ar;
        i = ai;
    }

    public static void main(String s[]) {
        NumeroComplexo x = new NumeroComplexo();
        x.atrib(1.2, 9.8);
        System.out.format("(%.1f + %.1fi)\n", x.r, x.i);
    }
}
```

# Classes e Tipos de Membros

- Tipos de variáveis e métodos declarados em uma classe:
  - variáveis e métodos da instância:
    - criadas na instanciação
    - não podem ser utilizadas sem uma instância
    - uma cópia por instância
  - variáveis e métodos da classe:
    - criadas na declaração da classe
    - estão sempre disponíveis
    - uma cópia por classe, todas as instâncias usam a mesma

# Método da Classe

- Veja classe HelloWorld

```
class HelloWorld {
   public static void main(String[] args) {
     System.out.println("Hello World!");
   }
}
```

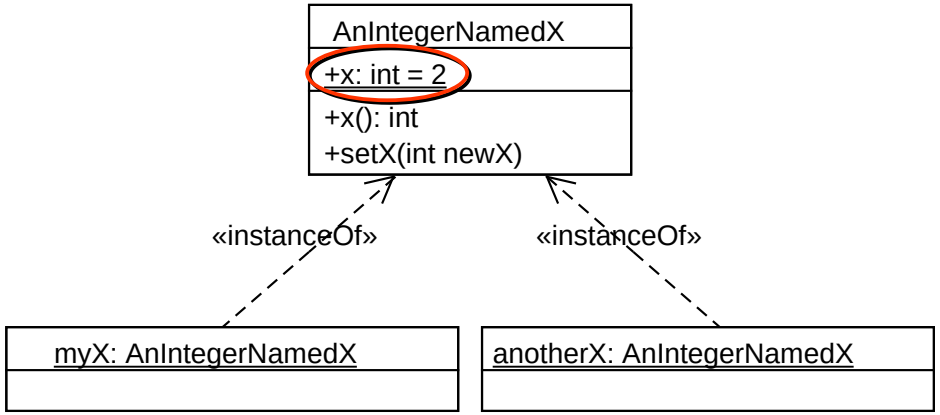
  - sendo *static*, o método *main* é um método da classe, ou seja, existe sempre após a declaração.

  - o modificador *static* torna o método *main* disponível antes da instanciação, dessa forma ele pode ser chamado já no início pelo SO

# Variáveis da Classe

```
class AnIntegerNamedX {
  public static int x;

  public int x() {
    return x;
  }

  public void setX(int newX) {
    x = newX;
  }
}

. . .

  AnIntegerNamedX myX = new AnIntegerNamedX();
  AnIntegerNamedX anotherX = new AnIntegerNamedX();
  myX.setX(1);          //ou  myX.x = 1;
  anotherX.setX(2);     //ou  anotherX.x = 2;
  System.out.println("myX.x = " + myX.x());
  System.out.println("anotherX.x = " + anotherX.x());
. . .
```

# Variáveis da Classe




```
class AnIntegerNamedX {
  public static int x;

  public int x() {
    return x;
  }

  public void setX(int newX) {
    x = newX;
  }
}

 . . .

  AnIntegerNamedX myX = new AnIntegerNamedX();
  AnIntegerNamedX anotherX = new AnIntegerNamedX();
  myX.setX(1);  ou  myX.x = 1;
  anotherX.setX(2);    ou  anotherX.x = 2;
  System.out.println("myX.x = " + myX.x());
  System.out.println("anotherX.x = " + anotherX.x());

. . .
```

No vídeo:

```
myX.x = 2
anotherX.x = 2
```

# Métodos da Classe

```
class AnIntegerNamedX {
  public static int x;

  static public int x() {
    return x;
  }

  static public void setX(int newX) {
   x = newX;
  }
}
  . . .
  AnIntegerNamedX.setX(3);
  System.out.println("AnIntegerNamedX.x = " +
                     AnIntegerNamedX.x());
  . . .
```

não é necessário instanciar

| AnIntegerNamedX |
| --- |
| +x: int = 3 |
| +x(): int<br>+setX(int newX) |

# Construtores

- Método especial usado para inicia um novo objeto.
  - é sempre o primeiro método a ser executado no objeto

```
class AnIntegerNamedX {
    int x;
    public int x() {
        return x;
    }
    public void setX(int newX) {
        x = newX;
    }
}
. . .
AnIntegerNamedX myX = new AnIntegerNamedX();
AnIntegerNamedX anotherX = new AnIntegerNamedX();
. . .
```

# Passagem de Parâmetros

- Os métodos podem receber parâmetros
  - Por valor
    - Apenas para tipos primitivos (int, float, long, double, boolean, char)
  - Por referência
    - Instâncias de classes (objetos) e vetores

# Declarando Métodos

- A classe **`Caneta`** implementa um método construtor

```
class Caneta {
    int r, g, b;
    public void Caneta(int ar, int ag, int ab) {
        r = ar; g = ag; b = ab;
    }
    public void leCaneta(CorRGB cor) {
        cor.red = r; cor.green = g; cor.blue = b;
    }
}
class CorRGB {
    public int red, green, blue;
}
```

Parâmetros por valor

Parâmetros por referência

- Um construtor tem o mesmo nome da classe e é executado no instante da instanciação.
- Este construtor é um método e pode conter passagem de parâmetros
  - No exemplo, por valor (no construtor)

# Declarando Métodos

- A classe abaixo implementa o método **`leCaneta()`**

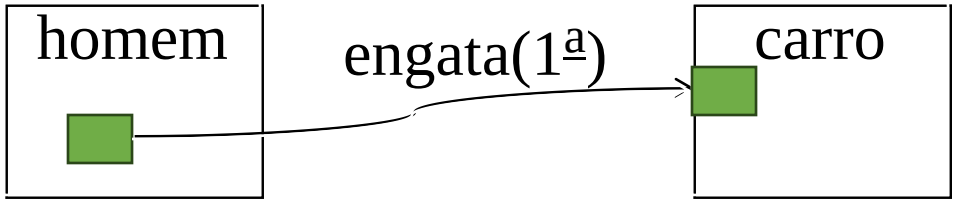
```
class Caneta {
    int r, g, b;
    public void Caneta(int ar, int ag, int ab) {
        r = ar; g = ag; b = ab;
    }
    public void leCaneta(CorRGB cor) {
        cor.red = r; cor.green = g; cor.blue = b;
    }
}
class CorRGB {
    public int red, green, blue;
}
...
CorRGB c = new CorRGB();
Caneta can = new Caneta(0, 0, 255);
can.leCaneta(c);
System.out.println("Cor: ("+c.red+","+c.green+","+c.blue+")");
...
```

- o método leCaneta() utiliza passagem de parâmetros por referência
  - As instâncias (criadas por new) são passadas por referência

# O que são Mensagens?

- Quando um objeto A "deseja" que o objeto B execute um método de B, o objeto A envia uma mensagem ao objeto B

exemplo:




- o número da marcha é o parâmetro do método engata.

# Mensagens

- Cada mensagem é composta pelo nome do objeto, nome da mensagem e argumentos
- mensagem significa chamada de um método do objeto destino
- exemplo:

```
obj.func(12);
```

- Mensagem func enviada ao objeto obj.

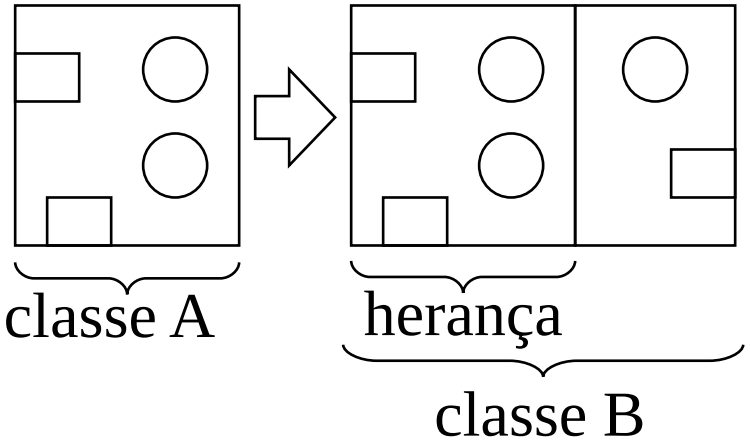
  Onde: obj ⇨ é o nome do objeto;

  func ⇨ é o método a ser executado e

  12 é o parâmetro.

# Que é Herança

- Uma classe pode ser definida a partir de características de outra classe (só uma)




A é dita ser superclasse de B

B é subclasse de A

- Vantagens:
  - subclasses com comportamento especializado das superclasses, facilidade para o reuso
  - criação de superclasses abstratas com comportamentos “genéricos”

# Override

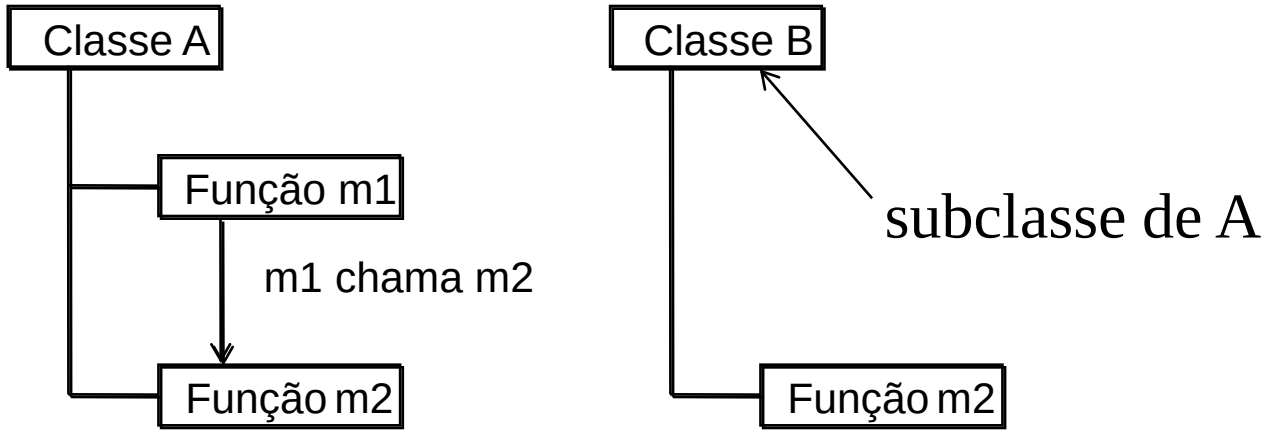
- Override é a capacidade de uma subclasse redefinir o corpo de um método implementado na superclasse o qual não tem o comportamento condizente com as necessidades da nova classe.

- Uma subclasse deve redefinir pelo menos todos os métodos declarados com *abstract* na superclasse.

- Não podemos redefinir um método declarado como final na superclasse.

# Override

- Considere uma instância de A (objA) e uma de B (objB). O que acontecerá com as mensagens objA.m1() e objB.m1()?




- objA.m1() ⇨ executa A.m1 que chama A.m2
- objB.m1() ⇨ executa A.m1 que chama B.m2

# Override

```
class A {
    public void m1() {
        System.out.println("A.M1");
        m2();
    }
    public void m2() {
        System.out.println("A.M2");
    }
}
class B extends A {
    public void m2() {
        System.out.println("B.M2");
    }
}
class PP {
    public static void main(String[] args) {
        A objA = new A();
        B objB = new B();
        objA.m1();
        objB.m1();
    }
}
```

# Override

```
class A {
    public void m1() {
        System.out.println("A.M1");
        m2();
    }
    public void m2() {
        System.out.println("A.M2");
    }
}
class B extends A {
    public void m2() {
        System.out.println("B.M2");
    }
}
class PP {
    public static void main(String[] args) {
        A objA = new A();
        B objB = new B();
        objA.m1();
        objB.m1();
    }
}
```

A superclasse de B

instanciações

mensagens

# Métodos e Variáveis

```
class UmaClasseBoba {
    boolean aVariavel;
    void aMetodo() {
      aVariavel = true;
    }
  }
  class UmaClasseBobona extends UmaClasseBoba {
    boolean aVariavel;
    void aMetodo() {
      aVariavel = false;
      super.aMetodo();
      System.out.println(aVariavel);
      System.out.println(super.aVariavel);
    }
  }
```

chama aMetodo da superclasse

variável da superclasse

- chamando aMetodo() de UmaClasseBobona, obtém-se:

```
false
true
```

- Os elementos de uma classe, mesmo com o nomes iguais, continuam existindo nas classes descendentes.

# Métodos e Variáveis

- Variável
  - Membro (disponível a todos métodos da classe);
  - Local (disponível no interior do seu método);
  - Não existem variáveis globais.

```
class UmaClasse {
    int variavelMembro;

    void metodoMembroUm() {
        int variavelLocal;
        //... Uso da variavelMembro e da variavelLocal
    }
    void metodoMembroDois() {
        int variavelLocal;
        //... Uso da variavelMembro e da variavelLocal
    }
}
```

Variáveis diferentes

# Sobrecarga

- Possibilidade de se atribuir mais que uma estratégia a um mesmo nome (método). Exemplo:

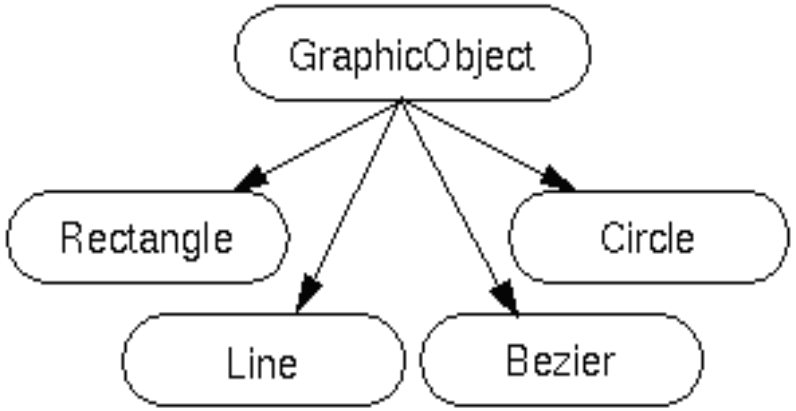
```
class Vetor2D {
    float x, y;
    Vetor2D() {
        x = 0; y = 0;
    }
    Vetor2D(float X, float Y) {
        x = X; y = Y;
    }
    ...
};
```

Neste exemplo a sobrecarga ocorre no construtor, mas ela pode aparecer em qualquer método.

- O compilador analisa o tipo dos argumentos e então escolhe qual método deve utilizar.

# Abstract

- Uma classe abstrata não pode ser instanciada, é preciso uma classe descendente.
- Um método abstrato deve ser redefinido (override) para poder ser chamado.



# Abstract

```
abstract class GraphicObject {
    int x, y;

    . . .

    void moveTo(int newX, int newY) {
        x = newX;
        y = newY;
        draw();
    }

    abstract void draw();
}
```

```
class Circle extends GraphicObject {
    void draw() {
        . . .
    }
}

class Rectangle extends GraphicObject {
    void draw() {
        . . .
    }
}
```

O abstract força que os métodos sejam redefinidos (override) na classe descendente.

# Interface

- Uma interface é uma coleção de declarações de métodos (sem implementações) e constantes.

- É similar a uma classe abstrata
  - os métodos devem ser redefinidos

- Limitações:
  - não é possível herdar variáveis de uma interface;
  - deve-se evitar implementações de métodos nas interfaces
    - mas é possível se o método tiver o modificador *default*.

# Interface

- A declaração de uma interface:

```
[public] interface InterfaceName [extends listOfSuperInterfaces] {
    . . .
}
```

uma interface pode ser descendente de outras interfaces (descendência múltipla), mas não pode ser descendente de uma classe.

- Exemplo:

```
interface Collection {
    final int MAXIMUM = 500;

    void add(Object obj);
    void delete(Object obj);
    Object find(Object obj);
    int currentCount();
}
```

# Declaração geral de uma Classe

- A declaração de uma classe tem a forma:

```
[modifiers] class ClassName [extends SuperClassName] [implements InterfaceNames] {
    . . .
}
```

- Aspectos definidos pela declaração da classe
  - *modifiers* ⇨ classe é *public*, *abstract*, ou *final*
  - *ClassName* ⇨ nome da classe
  - *SuperClassName* ⇨ nome da superclasse
  - *InterfaceNames* ⇨ lista separada por virgulas das interfaces implementadas pela ClassName
- Todas as classes são descendentes da classe Object (mesmo se omitida).

# Declaração geral de uma Variável

- A declaração de uma variáveis membro é:

```
[accessSpecifier] [static] [final] [volatile] type variableName
```

  - *accessSpecifier* ⇨ define qual outra classe tem acesso à variável (public, protected ou private)
  - *static* ⇨ indica que a variável é da classe
  - *final* ⇨ indica que a variável é uma constante
  - *volatile* ⇨ significa que a variável pode ser alterada assincronamente

- Exemplo de uma constante:

```
class Avo {
    final double AVOGADRO = 6.023e23;
}
```

# Declaração geral de um Método

- A declaração de um método é:

```
[accessSpecifier] [static] [abstract] [final] [native] [synchronized]
   returnType methodName ([paramlist]) [throws exceptionsList]
```

  - *abstract* ⇨ força a substituição do método na subclasse
  - *final* impede que o método seja substituído (override) em uma subclasse
  - *native* ⇨ permite que um método definido em outra linguagem possa ser utilizado em JAVA
  - *synchronized* ⇨ impede a execução do método por linhas concorrente (thread). Apenas uma linha pode executar por vez
  - *throws exceptionsList* ⇨ lança a exceção para o método chamador.

# Pacotes (Packages)

- Em JAVA uma biblioteca de classes é chamada de package. Pacotes podem conter também interfaces.
- Todas as classes pertencem a um pacote. Quando não especificado o pacote usado é o padrão (default).
- O nome de um pacote está diretamente relacionado ao nome do diretório onde está.
- JAVA utiliza a variável de ambiente CLASSPATH para encontrar os pacotes no disco.

# Pacotes

- Para utilizar classes e interfaces de um outro pacote:
  - é necessário importar (import) o pacote com as classes e interfaces;
  - somente as classes e interfaces públicas podem ser importadas;
  - quando várias classes de um mesmo pacote são importadas pode-se utiliza curingas (*);
  - o pacote padrão (sem nome) e o pacote *java.lang* são importados automaticamente

# Pacotes JAVA

- Diversos pacotes são fornecidos com o conjunto de desenvolvimento, entre eles:
  - java.lang ⇨ classes que compõem o núcleo da linguagem JAVA
  - java.io ⇨ classes de entrada/saída
  - java.awt ⇨ *Abstract Window Toolkit* classes para manipulação de janelas e controles
  - java.applet ⇨ criação de *applets*
  - java.net ⇨ classes para operação com redes
  - java.util ⇨ coleção de classes de utilidades (Dictionary, Stack, Vector, Hashtable)

# Pacotes JAVA

```
import java.io.File;

class Ren {
    public static void main(String[] args) {
        if (args.length != 2) {
            System.err.println(
                    "Usage: java Ren <old name> <new name>");
            System.exit(-1);
        }
        File f1 = new File(args[0]);
        File f2 = new File(args[1]);
        System.out.println(f1.getPath() +
(f1.renameTo(f2) ? " renamed to " : " could not be renamed to ") + f2.getPath());
    }
}
```

classe File do pacote java.io

# Controle de Acesso

- É possível restringir o acessos aos membros de uma classe

| Especificador | classe | subclasse | package | mundo |
|---|---|---|---|---|
| private | X | | | |
| protected | X | X | X | |
| public | X | X | X | X |
| package | X | | X | |

- exemplo:

```
class Restrita {
  private int dado;
  protected void rotina() {
    ...
  }
}
```

permite o acesso apenas pelos membros desta classe